Połączenie do bazy danych. Modele i relacje w EntityFrameworkCore

Programowanie aplikacji WWW w technologii .NET, 2024/2025

Przygotowała: I. Kartowicz-Stolarska

CEL

1. Tworzenie lokalnej bazy danych i podłączenia jej do projektu aplikacji WWW.
2. Tworzenie tabel w bazie danych według zasady code-first.
3. Normalizacja bazy danych i relacje między modelami EntityFrameworkCore.

PAKIETY NUGET

- Micorosoft.EntityFrameworkCore
- Microsoft.EntityFrameworkCore.SqlServer
- Microsoft.EntityFrameworkCore.Tools
- Microsoft.EntityFrameworkCore.Design

ROZSZERZENIA DO VS CODE

- https://marketplace.visualstudio.com/items?itemName=ms-mssql.mssql

TUTORIAL

Utwórz nowy projekt o nazwie SuperHeroes.

Instalacja pakietów

Z poziomu Projekt -> Zarządzanie projektami NuGet zainstaluj biblioteki Microsoft.EntityFrameworkCore:

- Microsoft.EntityFrameworkCore
- Microsoft.EntityFrameworkCore.SqlServer
- Microsoft.EntityFrameworkCore.Tools
- Microsoft.EntityFrameworkCore.Design

Pakiety Nuget możesz rownież zainstalować za pomocą wiersza poleceń np.

- dotnet add package Microsoft.EntityFrameworkCore

Metoda Code-first

1. W strukturze projektu utwórz nowy folder Models.
2. Do katalogu Models dodaj klasę
   a. *Hero.cs* o następującej strukturze:

```
public class Hero {
      public int Id { get; set; }
      public string Name { get; set; }
      public string Bio { get; set; }
      public string FirstName { get; set; }
      public string LastName { get; set; }
}
```

3. W katalogu projektu utwórz katalog Data z klasą kontekstu dla danych *PeopleContext.cs:*

```
public class HeroesContext : DbContext {
        public DbSet<Hero> Hero { get; set; }
    }
```

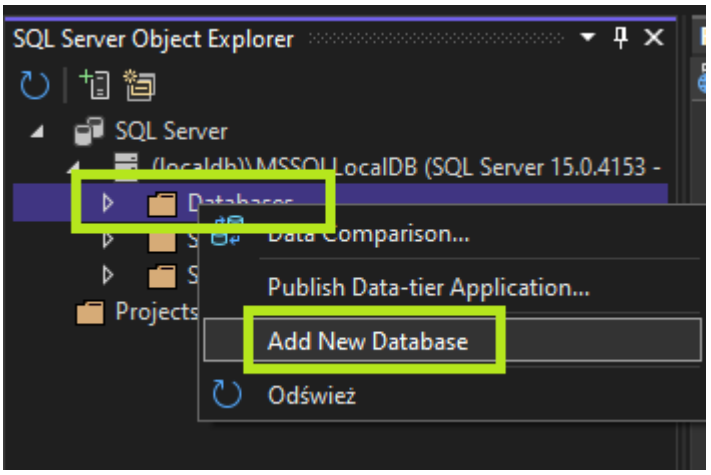
   Dodaj wymagane przestrzenie nazw:

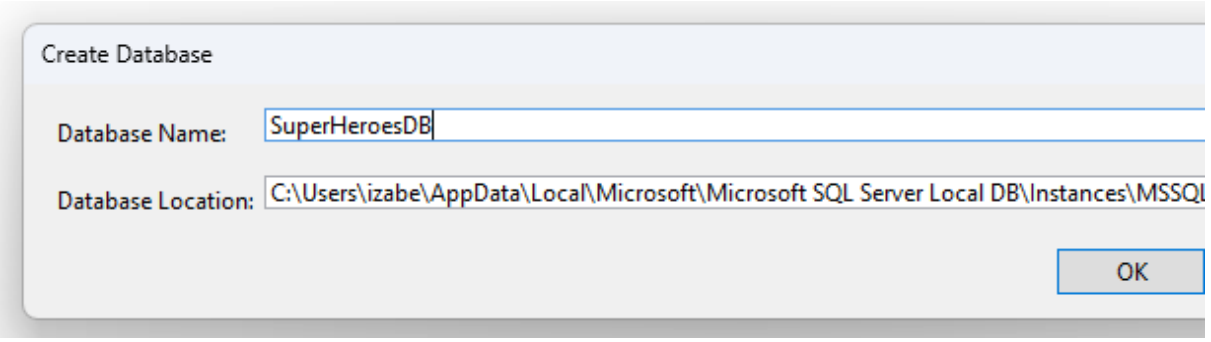
```
using SuperHeroes.Models;
using Microsoft.EntityFrameworkCore;
```

Tworzenie bazy danych

Domyślna nazwa serwera dla środowiska lokalnego: (localdb)\MSSQLLocalDB

1. Microsoft Visual Studio
   a. Utwórz nową bazę danych o nazwie EFDemoDB (Widok->SQL Server Object Explorer -> Databases -> Add new database)





Połączenie do bazy danych

2. Z właściwości bazy danych pobierz wartość ConnectionString i zapisz ją w `appsettings.json` w kluczu "ConnectionStrings" pod nazwą SuperHeroesDB:

```
{
….,
"ConnectionStrings": {
  "SuperHeroesDB": "Data Source=(localdb)\\MSSQLLocalDB;Initial
Catalog=EFDemoDB;Integrated Security=True;"
},
….
}
```

3. W pliku kontekstu danych HeroesContext.cs rozszerz konstruktor klasy o parameter options:

```
public class HeroesContext : DbContext {
      public HeroesContext(DbContextOptions options) :
base(options) { }

      public DbSet<Hero> Hero { get; set; }
}
```

4. W pliku Program.cs skonfiguruj połączenie z bazą SuperHeroesDB:

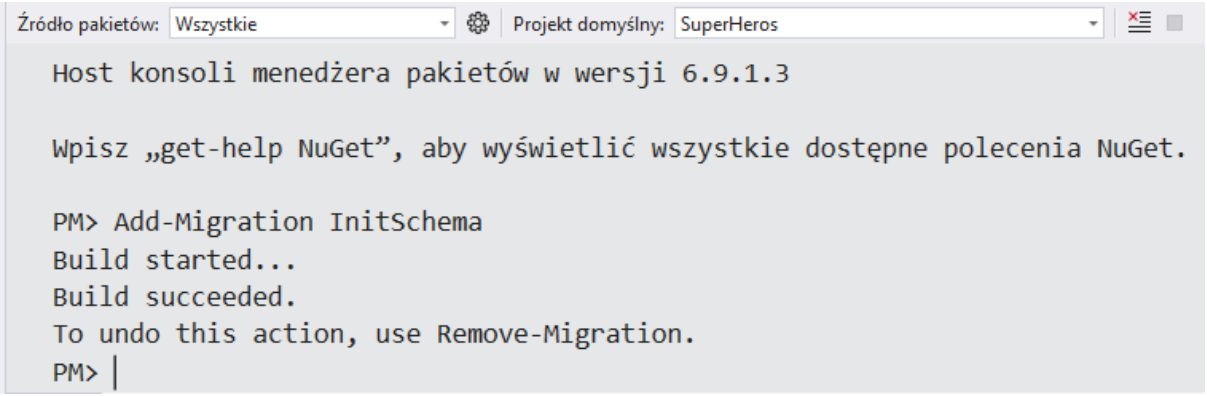
```
….
builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<HeroesContext>(options =>
      options.UseSqlServer(builder.Configuration.GetConnection
String("SuperHeroesDB")));

….
```

Dodaj do przestrzeni nazw:

- `using SuperHeroes.Data;`
- `using Microsoft.EntityFrameworkCore;`

Zarządzanie schematem danych za pomocą migracji

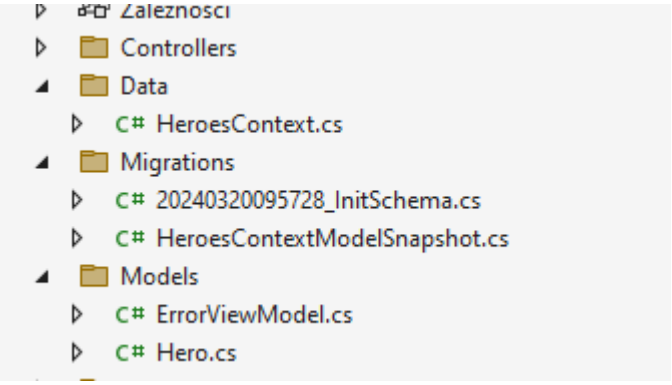
1. Uruchom konsolę menedżera pakietów (Widok->Inne okna->Konsola Menedżera pakietów)
2. W konsoli uruchom komendę tworzącą pierwszą migrację o nazwie InitSchema:
   `Add-Migration InitSchema`



Polecenie Add-Migration tworzy plik ze strukturą zmian bazy danych i zapisuje go pod podaną nazwą w katalogu Migrations. Katalog jest tworzony wraz z uruchomieniem tworzenia pierwszej migracji.

Aby użyć narzędzi EntityFrameworkCore w Visual Studio Code lub w konsoli należy w konsoli uruchomić polecenie instalujące komendy linii poleceń dla Entity Framework:

```
dotnet tool install --global dotnet-ef

dotnet ef migrations add InitSchema
```



3. Przeanalizuj utworzony plik migracji (funkcje Up i Down).
4. Za pomocą polecenia `Update-Database (lub dontnet ef database update)` zaktualizuj bazę danych.
5. Czy w bazie danych pojawiły się tabele? Jakie?
6. Zaktualizuj model Person.cs:

```
public class Hero {
      public int Id { get; set; }
      [Required]
      [Column(TypeName = "varchar(50)")]
      [MaxLength(100)]
      public string Name { get; set; }
      [MaxLength(256)]
      public string Bio { get; set; }
      [MaxLength(50)]
      public string FirstName { get; set; }
      [MaxLength(50)]
      public string LastName { get; set; }
}
```

7. Utwórz nową migrację o dowolnej nazwie np.
   `Add-Migration AddDataAnnotation`
   lub
   `dotnet ef migrations add nazwa`
8. Zaktualizuj bazę danych.
   `Update-Database`
   lub
   `dotnet ef database update`
9. Zaobserwuj zmiany w bazie danych.

Relacja jeden do wielu (OneToMany)

Jedna bohater może być w jednym zespole. Do jednego zespołu może należeć wielu bohaterów.

1. W katalogu Models dodaj klasę *Team.cs* o następującej strukturze:

```
public class Team {
      public int Id { get; set; }
      [Required]
      [MaxLength(50)]
      public string Name { get; set; }
}
```

2. Uzupełnij klasę `Hero.cs` o relację do modelu Team:

```
 public class Hero {
      ….
```

```
        public int TeamId { get; set; }
        public virtual Team Team { get; set; }
 }
```

3. Rozszerz model Team o kolejne właściwości:

```
public class Team
{
        …..
        public virtual ICollection<Hero>? Heroes { get; set; }
}
```

4. Dodaj mapowanie klasy *Team* na dataset *Team w* Data/*HeroesContext.cs*:

```
public class HeroesContext : DbContext
{
        ……
        public DbSet<Team> Team { get; set; }
}
```

5. Z poziomu terminala lub Konsoli Managera Pakietów utwórz migrację (Add-Migration). Przeanalizuj wygenerowany plik, a następnie uruchom komendę wygrywającą zmiany zapisane w migracji (Update-Database) do bazy danych.

Relacja wiele do wielu (ManyToMany)

Jeden superbohater pojawia się w wielu filmach. W jednym filmie może pojawić się wielu superbohaterów.

1. W katalogu *Models* utwórz nową klasę *Movie.cs* o następującej strukturze:

```
public class Movie {
        public int Id { get; set; }
        [Required]
        [MaxLength(100)]
        public string Title { get; set;}
}
```

2. Dodaj do klasy Movie.cs relację do modelu Hero.cs:

```
public class Movie
{
        ….
        public ICollection<Hero> Heros{ get; set; } = new
List<Hero>();
}
```

3. Dodaj do modelu Hero.cs relację do modelu Movie.cs

```
public class Hero {
        …..
        public ICollection<Movie> Movies { get; set; } = new
List<Movie>();
}
```

4. W Data/HeroesContext.cs i dodaj mapowanie na model Movie.cs oraz nową metodę OnModelCreating. W metodzie tej zostaną wskazane pola z tabeli pośredniczącej, które odpowiadają za stworzenie relacji wiele-do-wielu.

```
public class HeroesContext : DbContext
{
        …
        public DbSet<Movie> Movie { get; set; }

        protected override void OnModelCreating(ModelBuilder
modelBuilder)
        {
        modelBuilder.Entity<Hero>()
                .HasMany(e => e.Movies)
                .WithMany(e => e.Heros);
        }
}
```

5. Z poziomu terminala lub Konsoli Managera Pakietów utwórz migrację (Add-Migration). Przeanalizuj wygenerowany plik, a następnie uruchom komendę wygrywającą zmiany zapisane w migracji (Update-Database) do bazy danych.
6. Sprawdź jakie relacje zostały utworzone.

PRZYDATNE LINKI

- EntityFramework
- https://www.youtube.com/watch?v=rXP632YwwfQ
- https://www.youtube.com/watch?v=Qh2hgIc90y0
- Zaawansowane relacje ManyToMany
- https://channel9.msdn.com/Series/Entity-Framework-Core-101/Getting-Started-with-Entity-Framework-Core
- https://channel9.msdn.com/Series/Entity-Framework-Core-101/Performance-Tips
- https://channel9.msdn.com/Series/Entity-Framework-Core-101/Database-Providers
- Różnice między nvarchar a varchar
- Metoda Code-first
- Metoda Database-first
- Zapytania do bazy danych z poziomu Visual Studio Code

NARZĘDZIA

- SQL Server Express (instalacja Basic)
  https://www.microsoft.com/pl-pl/sql-server/sql-server-downloads
- SSMS - SQL Server Management Studio
  https://docs.microsoft.com/en-us/sql/ssms/download-sql-server-management-studio-ssms?view=sql-server-ver15